



# Stage 00 — Raw data preparation

S00\_raw\_data\_preparation.ipynb

## Resumen

Esta notebook realiza la **preparación inicial del dataset intradía del MNQ (Micro E-mini Nasdaq 100)**.

El objetivo es construir un dataset limpio, consistente y estructurado que servirá como base para la ingeniería de factores y el entrenamiento de modelos.

### 0. Configuración del entorno

- Clonado del repositorio y montaje de Google Drive.
- Instalación e importación de librerías necesarias.

### 1. Fuente de datos

- Datos históricos intradía del MNQ (OHLCV, frecuencia de 1 minuto) exportados desde NinjaTrader.
- Archivos originales en formato `.txt`, en zona horaria UTC.

### 2. Generación del dataset

- Unificación de todos los archivos `.txt` en un único DataFrame.
- Asignación de nombres de columnas: `open`, `high`, `low`, `close`, `volume`.
- Conversión de la columna `datetime` a índice temporal.

### 3. Filtrado

- Conserva solo **días hábiles bursátiles** (se eliminan fines de semana y feriados de mercado de EE.UU.).
- Conversión de marcas de tiempo de **UTC** → **US/Eastern**.
- Filtrado de **horario de mercado** (09:30–16:00) más pre-market (desde 08:30).

### 4. Validación de registros diarios

- Verificación de que cada día contenga la cantidad esperada de registros minuto a minuto.
- Detección y eliminación de días incompletos o con irregularidades.

### 5. Chequeo de continuidad temporal

- Confirmación de que los datos intradía estén en intervalos consecutivos de 1 minuto, sin gaps.

### 6. Dataset final

- Guardado del dataset limpio en formato `.parquet` dentro de Google Drive.

**Resultado:** Un dataset intradía del MNQ completamente limpio y estandarizado, listo para la ingeniería de factores y el modelado.

## Configuración del Entorno

### Importación de librerías

```
In [1]: # Utilidades generales
from datetime import datetime, timedelta
import os
import glob
import warnings
warnings.filterwarnings('ignore')

# Manejo y procesamiento de datos
import pandas as pd
from tabulate import tabulate
import pyarrow as pa
import pyarrow.parquet as pq

# Calendario de mercados
import pandas_market_calendars as mcal
import pandas as pd
import requests
from io import StringIO
```

### Rutas de archivos

```
In [2]: from pathlib import Path

# Buscar la raíz del proyecto
PROJECT_ROOT = Path.cwd()

while PROJECT_ROOT.name != "neural_profit_local":
    PROJECT_ROOT = PROJECT_ROOT.parent

print("Project root:", PROJECT_ROOT)
```

Project root: c:\Users\heguu\OneDrive\Escritorio\neural\_profit\_local

### Función para revisar información de dataset

```
In [3]: from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, Optional, Tuple

import pandas as pd

def mnq_dataset_info(
    df: pd.DataFrame,
    *,
    name: str = "mnq_raw",
    tz_assume_if_naive: Optional[str] = None, # ej: "UTC" o "America/New_York"
```

```

    day_def: str = "calendar", # "calendar" (fecha calendario) o "trading" (día)
) -> Dict[str, Any]:
    """
    Resume un dataset OHLCV con DatetimeIndex (ideal para mnq_raw).

    - Si el índice es tz-naive:
        - Si tz_assume_if_naive != None, lo localiza a esa tz.
        - Si no, reporta "tz-naive" (no se puede afirmar horario UTC).
    - Devuelve dict con métricas principales (y lo imprime bonito si se desea).
    """
    if not isinstance(df.index, pd.DatetimeIndex):
        raise TypeError(f"{name}: se requiere DatetimeIndex, recibido: {type(df.index)}")

    idx = df.index

    # --- timezone / UTC info ---
    tzinfo = idx.tz
    if tzinfo is None:
        tz_status = "tz-naive (sin zona horaria)"
        if tz_assume_if_naive:
            idx = idx.tz_localize(tz_assume_if_naive)
            tzinfo = idx.tz
            tz_status = f"localizado como {tzinfo}"
    else:
        tz_status = f"{tzinfo}"

    # --- rango temporal ---
    ts_min = idx.min()
    ts_max = idx.max()

    first_day = ts_min.date()
    last_day = ts_max.date()

    # --- días ---
    if day_def == "calendar":
        total_days = (pd.Timestamp(last_day) - pd.Timestamp(first_day)).days + 1
    elif day_def == "trading":
        total_days = idx.normalize().nunique()
    else:
        raise ValueError("day_def debe ser 'calendar' o 'trading'")

    # --- columnas ---
    columns = list(df.columns)

    # --- checks útiles ---
    n_rows = len(df)
    n_cols = df.shape[1]
    n_missing = int(df.isna().sum().sum())
    missing_by_col = df.isna().sum().to_dict()
    dup_index = int(idx.duplicated().sum())
    is_monotonic = bool(idx.is_monotonic_increasing)

    # Frecuencia estimada (puede fallar si hay huecos grandes)
    freq = pd.infer_freq(idx[: min(50000, len(idx))]) # muestra grande pero aco

    # Cobertura por día (min/max de hora del día, en tz del índice)
    # (Útil para ver si es 24/7 o horario de sesión)
    tod = pd.Series(idx.time)
    # Convertimos time a minutos del día para resumen robusto
    tod_minutes = pd.Series([t.hour * 60 + t.minute for t in tod])

```

```

typical_minute_min = int(tod_minutes.min())
typical_minute_max = int(tod_minutes.max())

# Rango promedio de filas por día (sólo días con datos)
rows_per_day = df.groupby(idx.normalize()).size()
rows_per_day_stats = {
    "days_with_data": int(rows_per_day.shape[0]),
    "rows_per_day_min": int(rows_per_day.min()),
    "rows_per_day_p50": float(rows_per_day.median()),
    "rows_per_day_max": int(rows_per_day.max()),
}

# Si hay tz, también mostramos rango en UTC
if tzinfo is not None:
    ts_min_utc = ts_min.tz_convert("UTC")
    ts_max_utc = ts_max.tz_convert("UTC")
    utc_range = (str(ts_min_utc), str(ts_max_utc))
    utc_note = "El índice está tz-aware; el horario UTC es inequívoco."
else:
    utc_range = None
    utc_note = "El índice es tz-naive; no se puede asegurar si está en UTC s

info: Dict[str, Any] = {
    "name": name,
    "shape": (n_rows, n_cols),
    "columns": columns,
    "index_type": type(df.index).__name__,
    "index_tz": tz_status,
    "utc_note": utc_note,
    "datetime_min": str(ts_min),
    "datetime_max": str(ts_max),
    "first_day": str(first_day),
    "last_day": str(last_day),
    "total_days": int(total_days),
    "day_definition": day_def,
    "utc_range_if_applicable": utc_range,
    "is_index_monotonic_increasing": is_monotonic,
    "duplicated_timestamps_in_index": dup_index,
    "inferred_freq_sample": freq,
    "missing_total_cells": n_missing,
    "missing_by_col": missing_by_col,
    "rows_per_day_stats": rows_per_day_stats,
    "time_of_day_minutes_range": {
        "min_minute_of_day": typical_minute_min,
        "max_minute_of_day": typical_minute_max,
    },
}
return info

def print_mnq_dataset_info(info: Dict[str, Any]) -> None:
    """Imprime el dict de mnq_dataset_info de forma ordenada."""
    print(f"Dataset: {info['name']}")
    print(f"Shape: {info['shape']}")
    print(f"Columns: {info['columns']}")
    print(f"Index: {info['index_type']} | TZ: {info['index_tz']}")
    print(f"Datetime min/max: {info['datetime_min']} -> {info['datetime_max']}")
    print(f"First/Last day: {info['first_day']} -> {info['last_day']}")
    print(f"Total days ({info['day_definition']}): {info['total_days']}")
    #print(f"Inferred freq (sample): {info['inferred_freq_sample']}")

```

```
#print(f"Index monotonic increasing: {info['is_index_monotonic_increasing']}")
#print(f"Duplicated timestamps in index: {info['duplicated_timestamps_in_ind']}")
#print(f"Missing total cells: {info['missing_total_cells']}")
#print(f"Missing by col: {info['missing_by_col']}")
#print(f"Rows/day stats: {info['rows_per_day_stats']}")
print(f"Time-of-day range (minutes): {info['time_of_day_minutes_range']}")
print(f"UTC note: {info['utc_note']}")
if info["utc_range_if_applicable"] is not None:
    print(f"UTC range: {info['utc_range_if_applicable'][0]} -> {info['utc_
```

# 1. Contexto y fuente de datos

Los datos corresponden al contrato MNQ (Micro E-mini Nasdaq 100) descargados desde NinjaTrader con frecuencia de un minuto (formato OHLCV).

- Open: precio de apertura
- High: precio máximo
- Low: precio mínimo
- Close: precio de cierre
- Volume: volumen negociado

Los datos están en la zona horaria UTC.

```
In [4]: SOURCE_PATH = PROJECT_ROOT / "data" / "00_source"
print(SOURCE_PATH)
txt_files = sorted(SOURCE_PATH.glob("*.txt"))
print(f"Archivos encontrados: {len(txt_files)}")
```

c:\Users\heguu\OneDrive\Escritorio\neural\_profit\_local\data\00\_source  
Archivos encontrados: 26

```
In [8]: import glob
import re
from pathlib import Path
import pandas as pd

# Mapeo de vencimientos trimestrales MNQ
MONTH_CODE_MAP = {
    "03": "H", # Marzo
    "06": "M", # Junio
    "09": "U", # Septiembre
    "12": "Z", # Diciembre
}

def extraer_contrato_mnq(nombre_archivo):
    """
    Extrae el contrato MNQ desde nombres como:

    00_mnq_03_20.Last
    01_mnq_06_20.Last
    02_mnq_09_20.Last
    03_mnq_12_20.Last

    Resultado:
    MNQH20, MNQM20, MNQU20, MNQZ20
    """
```

```

match = re.search(r"mnq_(03|06|09|12)_(\d{2})", nombre_archivo.lower())

if match is None:
    raise ValueError(f"No se pudo identificar el contrato en el archivo: {nombre_archivo}")

contract_month = match.group(1)
contract_year = match.group(2)

contract_code = MONTH_CODE_MAP[contract_month]

contract = f"MNQ{contract_code}{contract_year}"

return contract

def analizar_rangos_txt(source_path):

    ruta_historicos = f"{source_path}/*.txt"
    archivos = glob.glob(ruta_historicos)

    if not archivos:
        raise FileNotFoundError("No se encontraron archivos históricos.")

    resumen = []

    for archivo in sorted(archivos):

        nombre_archivo = Path(archivo).name

        # Extraer contrato desde el nombre del archivo
        contract = extraer_contrato_mnq(nombre_archivo)

        df = pd.read_csv(
            archivo,
            sep=";",
            header=None,
            usecols=[0], # SOLO datetime
            names=["datetime"]
        )

        df["datetime"] = pd.to_datetime(
            df["datetime"],
            format="%Y%m%d %H%M%S"
        )

        fecha_inicio = df["datetime"].min()
        fecha_fin = df["datetime"].max()
        n_rows = len(df)

        resumen.append({
            "archivo": nombre_archivo,
            "contract": contract,
            "fecha_inicio": fecha_inicio,
            "fecha_fin": fecha_fin,
            "n_rows": n_rows,
        })

    df_resumen = (
        pd.DataFrame(resumen)
    )

```

```

        .sort_values("fecha_inicio")
        .reset_index(drop=True)
    )

    return df_resumen

```

```

In [9]: df_rangos = analizar_rangos_txt(SOURCE_PATH)
df_rangos

```

```

Out[9]:

```

|    | archivo               | contract | fecha_inicio        | fecha_fin           | n_rows |
|----|-----------------------|----------|---------------------|---------------------|--------|
| 0  | 00_mnq_03_20.Last.txt | MNQH20   | 2019-12-23 03:01:00 | 2020-03-20 13:30:00 | 81660  |
| 1  | 01_mnq_06_20.Last.txt | MNQM20   | 2020-03-23 03:01:00 | 2020-06-19 13:30:00 | 86069  |
| 2  | 02_mnq_09_20.Last.txt | MNQU20   | 2020-06-22 03:01:00 | 2020-09-18 13:30:00 | 87482  |
| 3  | 03_mnq_12_20.Last.txt | MNQZ20   | 2020-09-21 03:01:00 | 2020-12-18 03:00:00 | 86824  |
| 4  | 04_mnq_03_21.Last.txt | MNQH21   | 2020-12-21 03:01:00 | 2021-03-19 13:30:00 | 84599  |
| 5  | 05_mnq_06_21.Last.txt | MNQM21   | 2021-03-22 03:01:00 | 2021-06-18 13:30:00 | 87090  |
| 6  | 06_mnq_09_21.Last.txt | MNQU21   | 2021-06-21 03:01:00 | 2021-09-17 13:30:00 | 88205  |
| 7  | 07_mnq_12_21.Last.txt | MNQZ21   | 2021-09-20 03:01:00 | 2021-12-17 14:30:00 | 88318  |
| 8  | 08_mnq_03_22.Last.txt | MNQH22   | 2021-12-20 03:01:00 | 2022-03-18 13:30:00 | 87112  |
| 9  | 09_mnq_06_22.Last.txt | MNQM22   | 2022-03-21 03:01:00 | 2022-06-17 13:30:00 | 87336  |
| 10 | 10_mnq_09_22.Last.txt | MNQU22   | 2022-06-20 03:01:00 | 2022-09-16 13:55:00 | 88207  |
| 11 | 11_mnq_12_22.Last.txt | MNQZ22   | 2022-09-19 03:01:00 | 2022-12-16 14:30:00 | 87830  |
| 12 | 12_mnq_03_23.Last.txt | MNQH23   | 2022-12-19 03:01:00 | 2023-03-17 13:30:00 | 85736  |
| 13 | 13_mnq_06_23.Last.txt | MNQM23   | 2023-03-20 03:01:00 | 2023-06-16 13:30:00 | 78856  |
| 14 | 14_mnq_09_23.Last.txt | MNQU23   | 2023-06-19 03:01:00 | 2023-09-15 13:30:00 | 88002  |
| 15 | 15_mnq_12_23.Last.txt | MNQZ23   | 2023-09-18 03:01:00 | 2023-12-15 14:30:00 | 88448  |
| 16 | 16_mnq_03_24.Last.txt | MNQH24   | 2023-12-18 03:01:00 | 2024-03-15 13:30:00 | 85777  |
| 17 | 17_mnq_06_24.Last.txt | MNQM24   | 2024-03-18 03:01:00 | 2024-06-21 13:30:00 | 93916  |
| 18 | 18_mnq_09_24.Last.txt | MNQU24   | 2024-06-24 03:01:00 | 2024-09-21 01:19:00 | 88216  |
| 19 | 19_mnq_12_24.Last.txt | MNQZ24   | 2024-09-23 03:01:00 | 2024-12-20 21:30:00 | 88498  |
| 20 | 20_mnq_03_25.Last.txt | MNQH25   | 2024-12-24 03:01:00 | 2025-03-21 13:30:00 | 83718  |
| 21 | 21_mnq_06_25.Last.txt | MNQM25   | 2025-04-06 08:42:00 | 2025-06-20 13:30:00 | 72228  |
| 22 | 22_mnq_09_25.Last.txt | MNQU25   | 2025-06-23 03:01:00 | 2025-09-19 13:30:00 | 82697  |
| 23 | 23_mnq_12_25.Last.txt | MNQZ25   | 2025-09-22 03:01:00 | 2025-12-19 14:30:00 | 86472  |
| 24 | 24_mnq_03_26.Last.txt | MNQH26   | 2025-12-22 03:01:00 | 2026-03-20 13:31:00 | 82549  |
| 25 | 25_mnq_06_26.Last.txt | MNQM26   | 2026-03-23 03:01:00 | 2026-04-17 20:18:00 | 26795  |

```

In [10]: import glob
import os

```

```

import pandas as pd

def analizar_superposicion_txt(source_path, filtro_especial="06_25"):

    ruta_historicos = f"{source_path}/*.txt"
    archivos = sorted(glob.glob(ruta_historicos))

    if not archivos:
        raise FileNotFoundError("No se encontraron archivos históricos.")

    resumen = []

    # =====
    # 1. Leer rango de cada archivo
    # =====
    for archivo in archivos:
        df = pd.read_csv(
            archivo,
            sep=";",
            header=None,
            usecols=[0],
            names=["datetime"]
        )

        df["datetime"] = pd.to_datetime(df["datetime"], format="%Y%m%d %H%M%S")

        resumen.append({
            "archivo": os.path.basename(archivo),
            "ruta": archivo,
            "fecha_inicio": df["datetime"].min(),
            "fecha_fin": df["datetime"].max(),
            "n_rows": len(df),
            "archivo_especial": filtro_especial in os.path.basename(archivo)
        })

    df_rangos = pd.DataFrame(resumen).sort_values(["fecha_inicio", "fecha_fin"])

    # =====
    # 2. Buscar superposición entre archivos
    # =====
    superposiciones = []

    for i in range(len(df_rangos)):
        archivo_i = df_rangos.loc[i]

        for j in range(i + 1, len(df_rangos)):
            archivo_j = df_rangos.loc[j]

            # Si el siguiente empieza después de que termina el actual,
            # ya no puede superponerse con este ni con los siguientes
            if archivo_j["fecha_inicio"] > archivo_i["fecha_fin"]:
                break

            # Verificar intersección
            inicio_solape = max(archivo_i["fecha_inicio"], archivo_j["fecha_inicio"])
            fin_solape = min(archivo_i["fecha_fin"], archivo_j["fecha_fin"])

            if inicio_solape <= fin_solape:
                superposiciones.append({
                    "archivo_1": archivo_i["archivo"],

```



```
        "archivo_2": archivo_j["archivo"],
        "inicio_solape": inicio_solape,
        "fin_solape": fin_solape,
        "duracion_solape": fin_solape - inicio_solape,
        "archivo_1_especial": archivo_i["archivo_especial"],
        "archivo_2_especial": archivo_j["archivo_especial"]
    })

df_super = pd.DataFrame(superposiciones)

# =====
# 3. Filtrar superposiciones donde aparezca 06_25
# =====
if not df_super.empty:
    df_super_0625 = df_super[
        (df_super["archivo_1_especial"]) | (df_super["archivo_2_especial"])
    ].copy()
else:
    df_super_0625 = pd.DataFrame()

return df_rangos, df_super, df_super_0625
```

```
In [11]: df_rangos, df_super, df_super_0625 = analizar_superposicion_txt(SOURCE_PATH, fil
```

```
In [12]: df_rangos
```

Out[12]:

|    |                       | archivo                                           | ruta | fecha_inicio           | fect      |
|----|-----------------------|---------------------------------------------------|------|------------------------|-----------|
| 0  | 00_mnq_03_20.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2019-12-23<br>03:01:00 | 202<br>13 |
| 1  | 01_mnq_06_20.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2020-03-23<br>03:01:00 | 202<br>13 |
| 2  | 02_mnq_09_20.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2020-06-22<br>03:01:00 | 202<br>13 |
| 3  | 03_mnq_12_20.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2020-09-21<br>03:01:00 | 202<br>03 |
| 4  | 04_mnq_03_21.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2020-12-21<br>03:01:00 | 202<br>13 |
| 5  | 05_mnq_06_21.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2021-03-22<br>03:01:00 | 202<br>13 |
| 6  | 06_mnq_09_21.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2021-06-21<br>03:01:00 | 202<br>13 |
| 7  | 07_mnq_12_21.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2021-09-20<br>03:01:00 | 202<br>14 |
| 8  | 08_mnq_03_22.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2021-12-20<br>03:01:00 | 202<br>13 |
| 9  | 09_mnq_06_22.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2022-03-21<br>03:01:00 | 202<br>13 |
| 10 | 10_mnq_09_22.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2022-06-20<br>03:01:00 | 202<br>13 |
| 11 | 11_mnq_12_22.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2022-09-19<br>03:01:00 | 202<br>14 |
| 12 | 12_mnq_03_23.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2022-12-19<br>03:01:00 | 202<br>13 |
| 13 | 13_mnq_06_23.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2023-03-20<br>03:01:00 | 202<br>13 |
| 14 | 14_mnq_09_23.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... |      | 2023-06-19<br>03:01:00 | 202<br>13 |

|    | archivo               | ruta                                              | fecha_inicio           | fect      |
|----|-----------------------|---------------------------------------------------|------------------------|-----------|
| 15 | 15_mnq_12_23.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2023-09-18<br>03:01:00 | 202<br>14 |
| 16 | 16_mnq_03_24.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2023-12-18<br>03:01:00 | 202<br>13 |
| 17 | 17_mnq_06_24.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2024-03-18<br>03:01:00 | 202<br>13 |
| 18 | 18_mnq_09_24.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2024-06-24<br>03:01:00 | 202<br>01 |
| 19 | 19_mnq_12_24.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2024-09-23<br>03:01:00 | 202<br>21 |
| 20 | 20_mnq_03_25.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2024-12-24<br>03:01:00 | 202<br>13 |
| 21 | 21_mnq_06_25.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2025-04-06<br>08:42:00 | 202<br>13 |
| 22 | 22_mnq_09_25.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2025-06-23<br>03:01:00 | 202<br>13 |
| 23 | 23_mnq_12_25.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2025-09-22<br>03:01:00 | 202<br>14 |
| 24 | 24_mnq_03_26.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2025-12-22<br>03:01:00 | 202<br>13 |
| 25 | 25_mnq_06_26.Last.txt | c:\Users\heguu\OneDrive\Escritorio\neural_prof... | 2026-03-23<br>03:01:00 | 202<br>20 |

In [13]: df\_super

Out[13]: —

## 2. Generación de dataset desde archivos históricos

Dado que los contratos se encuentran almacenados en archivos .txt dentro de la carpeta historicos\_mnq, es necesario unificarlos en un único dataset consolidado.

La siguiente función se encarga de leer los archivos .txt, asignar nombres a las columnas correspondientes y establecer la columna datetime como índice temporal del dataframe.

```
In [25]: import glob
import re
from pathlib import Path
import pandas as pd

# Mapeo de vencimientos trimestrales MNQ
MONTH_CODE_MAP = {
    "03": "H", # Marzo
    "06": "M", # Junio
    "09": "U", # Septiembre
    "12": "Z", # Diciembre
}

def extraer_contrato_mnq(nombre_archivo):
    """
    Extrae el contrato desde nombres como:

    00_mnq_03_20.Last.txt
    01_mnq_06_20.Last.txt
    02_mnq_09_20.Last.txt
    03_mnq_12_20.Last.txt

    Resultado:
    H20, M20, U20, Z20
    """

    match = re.search(
        r"mnq_(03|06|09|12)_(\d{2})",
        nombre_archivo.lower()
    )

    if match is None:
        raise ValueError(
            f"No se pudo identificar el contrato en el archivo: {nombre_archivo}"
        )

    month_num = match.group(1)
    year_short = match.group(2)

    contract_code = MONTH_CODE_MAP[month_num]
    contract = f"{contract_code}{year_short}"

    return contract

def generar_df():
    # Ruta a los archivos .txt
    ruta_historicos_drive = f"{SOURCE_PATH}/*.txt"

    # Determinar qué ruta usar
    if glob.glob(ruta_historicos_drive):
        print(f"Usando históricos desde {SOURCE_PATH}")
        ruta_historicos = ruta_historicos_drive
    else:
```

```

    raise FileNotFoundError(
        f"No se encontraron archivos históricos {SOURCE_PATH}."
    )

# Lista para almacenar DataFrames individuales
df_mnq = []

# Leer todos los archivos .txt
for archivo in sorted(glob.glob(ruta_historicos)):

    nombre_archivo = Path(archivo).name

    # Extraer contrato desde el nombre del archivo
    contract = extraer_contrato_mnq(nombre_archivo)

    df = pd.read_csv(
        archivo,
        sep=";",
        header=None,
        names=["datetime", "open", "high", "low", "close", "volume"],
        dtype={
            "open": float,
            "high": float,
            "low": float,
            "close": float,
            "volume": int,
        }
    )

    # Convertir columna datetime al formato datetime real
    df["datetime"] = pd.to_datetime(
        df["datetime"],
        format="%Y%m%d %H%M%S"
    )

    # Agregar contrato
    df["contract"] = contract

    # Establecer datetime como índice
    df.set_index("datetime", inplace=True)

    df_mnq.append(df)

# Unir todos los DataFrames
df_mnq_raw = pd.concat(df_mnq)

# Ordenar por fecha
df_mnq_raw.sort_index(inplace=True)

return df_mnq_raw

```

El siguiente bloque de código verifica si el dataset consolidado ya ha sido generado previamente.

En particular, comprueba la existencia del archivo mnq\_raw.parquet.

- Si el archivo está presente, se carga directamente en la variable df\_mnq.

- En caso contrario, se invoca la función `generate_dataset()` para generar el dataset a partir de los archivos originales.

```
In [26]: RAW_PATH = PROJECT_ROOT / "data" / "01_raw"
RAW_PATH.mkdir(parents=True, exist_ok=True)
```

```
In [27]: import os
import pandas as pd

def load_or_build_raw_dataset():

    raw_dir = f"{RAW_PATH}"
    mnq_raw_data_file = f"{RAW_PATH}/mnq_raw.parquet"

    # Crear carpeta si no existe
    os.makedirs(raw_dir, exist_ok=True)

    if os.path.exists(mnq_raw_data_file):
        print(f"Archivo encontrado en disco. Cargando dataset local desde {mnq_r
df_mnq_raw = pd.read_parquet(mnq_raw_data_file)

    else:
        print("Archivo no encontrado. Generando dataset desde archivos histórico
df_mnq_raw = generar_df()
df_mnq_raw.to_parquet(mnq_raw_data_file, index=True)
        print(f"Dataset generado y guardado localmente en {mnq_raw_data_file}.")

    return df_mnq_raw
```

```
In [28]: df_mnq_raw = load_or_build_raw_dataset()
```

Archivo no encontrado. Generando dataset desde archivos históricos...  
Usando históricos desde c:\Users\heguu\OneDrive\Escritorio\neural\_profit\_local\data\00\_source  
Dataset generado y guardado localmente en c:\Users\heguu\OneDrive\Escritorio\neural\_profit\_local\data\01\_raw\mnq\_raw.parquet.

```
In [29]: df_mnq_raw
```

Out[29]:

|                     | open     | high     | low      | close    | volume | contract |
|---------------------|----------|----------|----------|----------|--------|----------|
| datetime            |          |          |          |          |        |          |
| 2019-12-23 03:01:00 | 8718.50  | 8718.75  | 8718.50  | 8718.50  | 9      | H20      |
| 2019-12-23 03:02:00 | 8718.25  | 8718.25  | 8718.00  | 8718.25  | 14     | H20      |
| 2019-12-23 03:03:00 | 8718.25  | 8718.50  | 8718.00  | 8718.25  | 74     | H20      |
| 2019-12-23 03:04:00 | 8718.25  | 8719.00  | 8718.25  | 8718.50  | 10     | H20      |
| 2019-12-23 03:05:00 | 8718.50  | 8719.00  | 8718.50  | 8719.00  | 6      | H20      |
| ...                 | ...      | ...      | ...      | ...      | ...    | ...      |
| 2026-04-17 20:14:00 | 26835.50 | 26844.50 | 26835.50 | 26839.25 | 1120   | M26      |
| 2026-04-17 20:15:00 | 26839.50 | 26842.25 | 26837.75 | 26840.25 | 605    | M26      |
| 2026-04-17 20:16:00 | 26840.75 | 26841.75 | 26835.75 | 26841.25 | 615    | M26      |
| 2026-04-17 20:17:00 | 26840.50 | 26844.75 | 26840.00 | 26842.25 | 410    | M26      |
| 2026-04-17 20:18:00 | 26842.50 | 26843.75 | 26839.50 | 26840.25 | 221    | M26      |

2172640 rows × 6 columns

### 3. Verificación de duplicados

In [30]:

```
# =====
# VERIFICACIÓN 1: DUPLICADOS EXACTOS DE TIMESTAMP
# =====
# Detecta registros que comparten exactamente el mismo índice datetime.
# Esto permite identificar barras repetidas que podrían haberse cargado
# más de una vez durante el proceso de consolidación de archivos.

duplicados = df_mnq_raw.index.duplicated(keep=False)

# DataFrame con todos los registros involucrados en duplicados
df_dup = df_mnq_raw[duplicados]

print(f"Total registros duplicados: {duplicados.sum()}")

# Visualizar primeros duplicados encontrados
df_dup.head()
```

Total registros duplicados: 0

Out[30]:

|          | open | high | low | close | volume | contract |
|----------|------|------|-----|-------|--------|----------|
| datetime |      |      |     |       |        |          |

In [31]:

```
# =====
# VERIFICACIÓN 2: CANTIDAD DE REGISTROS POR DÍA
# =====
# Analiza cuántas barras existen por jornada de trading.
# Días con una cantidad anormalmente alta de registros pueden indicar
# duplicaciones parciales o problemas de carga de datos.
```

```

df_tmp = df_mnq_raw.copy()

# Extraer únicamente la fecha
df_tmp["date"] = df_tmp.index.date

# Contar registros por día
conteo_por_dia = df_tmp.groupby("date").size()

# Umbral de detección de días sospechosos
# Para MNQ intradía normalmente se esperan aproximadamente
# entre 390 y 450 registros por sesión.
dias_sospechosos = conteo_por_dia[conteo_por_dia > 500]

# Mostrar días con exceso de registros
dias_sospechosos.sort_values(ascending=False)

```

```

Out[31]: date
2025-01-30    1382
2025-11-19    1382
2025-06-11    1381
2026-01-12    1381
2024-05-06    1381
...
2025-07-15     668
2021-06-18     653
2020-03-16     642
2020-06-19     576
2020-10-23     576
Length: 1606, dtype: int64

```

```

In [32]: # =====
# VERIFICACIÓN 3: DUPLICADOS REALES POR DÍA
# =====
# Identifica timestamps repetidos dentro de una misma fecha.
# Esto permite localizar exactamente qué barras fueron duplicadas.

df_tmp = df_mnq_raw.copy()
df_tmp["date"] = df_tmp.index.date

duplicados_por_dia = (
    df_tmp
    .reset_index()
    .groupby(["date", "datetime"])
    .size()
    .reset_index(name="count")
)

# Mantener únicamente timestamps repetidos
duplicados_reales = duplicados_por_dia[
    duplicados_por_dia["count"] > 1
]

print(f"Total timestamps duplicados: {len(duplicados_reales)}")

duplicados_reales.head()

```

Total timestamps duplicados: 0



Out[32]: **date datetime count**

```
In [33]: # =====
# VERIFICACIÓN 4: RESUMEN DEL PROBLEMA DE DUPLICADOS
# =====
# Resume el alcance del problema detectado en las verificaciones anteriores.
#
# - Timestamps duplicados: cantidad de instantes únicos que aparecen más de
#   una vez dentro del dataset.
#
# - Días afectados: cantidad de jornadas que contienen al menos un timestamp
#   duplicado.
#
# Esta información permite determinar si el problema está concentrado en unas
# pocas fechas específicas o distribuido a lo largo de todo el histórico.

n_timestamps_duplicados = duplicados_reales.shape[0]
n_dias_afectados = duplicados_reales["date"].nunique()

print(f"Timestamps duplicados : {n_timestamps_duplicados}")
print(f"Días afectados          : {n_dias_afectados}")
```

Timestamps duplicados : 0  
Días afectados : 0

In [34]: df\_mnq\_raw.head()

Out[34]:

|                            | open    | high    | low     | close   | volume | contract |
|----------------------------|---------|---------|---------|---------|--------|----------|
| <b>datetime</b>            |         |         |         |         |        |          |
| <b>2019-12-23 03:01:00</b> | 8718.50 | 8718.75 | 8718.50 | 8718.50 | 9      | H20      |
| <b>2019-12-23 03:02:00</b> | 8718.25 | 8718.25 | 8718.00 | 8718.25 | 14     | H20      |
| <b>2019-12-23 03:03:00</b> | 8718.25 | 8718.50 | 8718.00 | 8718.25 | 74     | H20      |
| <b>2019-12-23 03:04:00</b> | 8718.25 | 8719.00 | 8718.25 | 8718.50 | 10     | H20      |
| <b>2019-12-23 03:05:00</b> | 8718.50 | 8719.00 | 8718.50 | 8719.00 | 6      | H20      |

## 4. Generación y guardado mnq\_raw\_summary.json

```
In [35]: # =====
# RESUMEN FINAL DEL DATASET INTRADÍA
# =====
# Objetivo:
# Generar un resumen descriptivo del dataset procesado y almacenarlo en
# formato JSON para su reutilización en etapas posteriores del proyecto.
#
# El archivo generado contiene información sobre:
# - Cobertura temporal.
# - Zona horaria.
# - Cantidad de registros.
# - Cantidad de días de trading.
```

```
# - Rango horario.
# - Estructura del dataset.
#
# Salida:
#   data/01_stage/mnq_intraday_summary.json
# =====

from pathlib import Path
import json

# Ruta de SUMMARY
MNQ_RAW_SUMMARY = RAW_PATH / "mnq_raw_summary.json"

# Generar resumen del dataset final
info_raw_final = mnq_dataset_info(
    df_mnq_raw,
    name="mnq_raw",
    tz_assume_if_naive="UTC",
    day_def="trading"
)

# Mostrar resumen por pantalla
print_mnq_dataset_info(info_raw_final)

# -----
# Guardar resumen en formato JSON
# -----

# Crear carpeta destino si no existe
MNQ_RAW_SUMMARY.parent.mkdir(
    parents=True,
    exist_ok=True
)

# Guardar archivo JSON
with MNQ_RAW_SUMMARY.open(
    "w",
    encoding="utf-8"
) as f:
    json.dump(
        info_raw_final,
        f,
        indent=2,
        ensure_ascii=False
    )

# Mostrar ubicación del archivo generado
print("\nResumen guardado correctamente:")
print(MNQ_RAW_SUMMARY.resolve())
```

```
Dataset: mnq_raw
Shape: (2172640, 6)
Columns: ['open', 'high', 'low', 'close', 'volume', 'contract']
Index: DatetimeIndex | TZ: localizado como UTC
Datetime min/max: 2019-12-23 03:01:00+00:00 -> 2026-04-17 20:18:00+00:00
First/Last day: 2019-12-23 -> 2026-04-17
Total days (trading): 2005
Time-of-day range (minutes): {'min_minute_of_day': 0, 'max_minute_of_day': 1439}
UTC note: El índice está tz-aware; el horario UTC es inequívoco.
UTC range: 2019-12-23 03:01:00+00:00 -> 2026-04-17 20:18:00+00:00

Resumen guardado correctamente:
C:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\01_raw\mnq_raw_summary.json
```